

IPVC — ESTG

Engenharia da Computação Gráfica e Multimédia

Unidade Curricular: Engenharia de Software

TaskTide

Gestor de Tarefas para Estudantes Universitários

Relatório de Projeto — Vibe Coding e CI/CD

Autores:

David Malafaya (N.º 29667)

João Rodrigues (N.º 29653)

Rita Miranda (N.º 34581)

Docente: Vasco Miranda

Ano letivo: 2025/2026 — 2.º Semestre

Junho de 2026

Índice

Índice.....	2
1. Introdução	3
2. Tema — Scholar OS.....	3
2.1. Descrição da Solução	3
2.2. Funcionalidades Principais	3
2.3. Arquitetura do Projeto	4
2.4. Stack Tecnológica.....	4
3. Plataforma Escolhida e Limitações	6
3.1. Lovable.dev — Ferramenta de Vibe Coding	6
Vantagens.....	6
Dificuldades	6
3.2. Exemplos de Prompts Utilizados	7
3.3. Limitações das Plataformas e da Integração CI/CD	7
4. GitHub, Agile e CI/CD.....	8
4.1. GitHub — Versionamento e Colaboração	8
4.2. Gestão Agile / Scrum — GitHub Projects.....	8
4.3. Product Backlog	Erro! Marcador não definido.
4.4. Pipeline de CI/CD.....	9
CI — GitHub Actions	9
CD — Vercel.....	9
5. Prints e Evidências	10
5.1. Aplicação em Funcionamento	10
5.2. GitHub — Repositório e Colaboração	11
5.3. Scrum Board — GitHub Projects.....	12
5.4. Pipeline CI/CD — GitHub Actions e Vercel	12
5.5. Links do Projeto	13
6. Conclusão e Análise Crítica	14
6.1. Processo e Metodologias de Desenvolvimento	14
6.2. Potencial das Ferramentas de IA	14
6.3. Limitações Encontradas.....	14
6.4. Vantagens do Trabalho Colaborativo	14
6.5. Impacto do CI/CD no Desenvolvimento	15
7. Bibliografia	16

1. Introdução

O presente relatório documenta o projeto TaskTide, uma aplicação web de gestão de tarefas orientada a estudantes universitários, desenvolvida no âmbito da atividade de Vibe Coding e CI/CD da unidade curricular de Engenharia de Software do curso de Engenharia da Computação Gráfica e Multimédia (IPVC-ESTG, 2025/2026).

O projeto teve como principais objetivos demonstrar a aplicação de processos modernos de Engenharia de Software, integrando ferramentas de Inteligência Artificial na prototipagem e geração de código (vibe coding), juntamente com práticas de versionamento com GitHub, gestão Agile/Scrum e um pipeline de CI/CD com deploy automatizado.

A solução desenvolvida permite que os estudantes organizem as suas tarefas académicas por cadeira, definam prazos e prioridades, e acompanhem o estado de progresso de cada item — tudo de forma local, sem necessidade de conta ou servidor, através de persistência em localStorage.

2. Tema — TaskTide

2.1. Descrição da Solução

O TaskTide é uma aplicação web single-page de gestão de tarefas académicas, focada em estudantes universitários. A aplicação apresenta uma interface limpa e estruturada, inspirada numa estética de "arquivo académico", com suporte bilingue (Português / Inglês) e funcionamento completamente offline.

O valor gerado pela aplicação reside na centralização da gestão de tarefas por cadeira curricular num único painel visual, sem dependências externas (sem login, sem servidor, sem subscrição). As tarefas podem ser criadas rapidamente através de um campo de adição rápida no cabeçalho, e geridas com prioridade (Alta, Média, Baixa), prazo, estado (A Fazer / Em Curso / Concluído) e cadeira associada.

2.2. Funcionalidades Principais

- Criação, edição e eliminação de cadeiras (Courses) com código curto gerado automaticamente (ex.: ES, PROG, CALC).
- Criação, edição, eliminação e filtragem de tarefas por cadeira selecionada.
- Estado de tarefa cíclico: A Fazer → Em Curso → Concluído (via clique no ícone de estado).
- Painel de "Próximas Datas" com prazos ordenados cronologicamente, com etiquetas dinâmicas "Hoje" e "Amanhã".
- Adição rápida de tarefas pelo cabeçalho (campo de texto com Enter).
- Toggle de idioma PT/EN com preferência persistida em localStorage.
- Persistência automática dos dados localmente no browser (localStorage).
- Interface responsiva e acessível em dispositivos móveis.

2.3. Arquitetura do Projeto

A aplicação foi construída com uma arquitetura frontend de página única (SPA), sem backend dedicado. A estrutura de ficheiros principal é a seguinte:

Ficheiro / Pasta	Responsabilidade
src/routes/index.tsx	Dashboard principal — orquestra todos os componentes
src/components/CourseSidebar.tsx	Sidebar com lista de cadeiras e toggle de idioma
src/components/TaskList.tsx	Lista de tarefas com ações de estado, edição e eliminação
src/components/TaskDialog.tsx	Diálogo modal de edição de tarefa (prazo, prioridade, estado)
src/components/UpcomingPanel.tsx	Painel lateral de próximos prazos ordenados
src/lib/storage.ts	Hook useStore() — CRUD + persistência em localStorage
src/lib/i18n.tsx	Dicionário PT/EN + LangProvider + hook useT()
src/styles.css	Tokens de design (cores, tipografia, variáveis CSS)

2.4. Stack Tecnológica

Tecnologia	Versão / Papel
React	v19 — biblioteca de interface reativa
TypeScript	v5.8 — tipagem estática
Vite	v8 — bundler e servidor de desenvolvimento
TanStack Router	v1.168 — routing file-based
Tailwind CSS	v4 — estilização utility-first
shadcn/ui + Radix UI	Componentes acessíveis (dialog, popover, calendar, etc.)
date-fns	v4 — formatação e cálculo de datas
lucide-react	v0.575 — ícones SVG
Lovable.dev	Plataforma de vibe coding (geração de código por IA)

GitHub	Versionamento e colaboração
GitHub Projects	Gestão Agile/Scrum (backlog, sprints, issues)
GitHub Actions	Pipeline de CI (lint + build automático)
Vercel	Deploy automático (CD) — hosting do protótipo online

3. Plataforma Escolhida e Limitações

3.1. Lovable.dev — Ferramenta de Vibe Coding

A plataforma principal utilizada para o desenvolvimento do protótipo foi o Lovable.dev, uma ferramenta de geração de código assistida por IA, especialmente vocacionada para aplicações web com React, Tailwind CSS e TypeScript. O Lovable permite descrever funcionalidades em linguagem natural (prompts) e gera código funcional pronto a executar, integrado com um repositório GitHub criado automaticamente.

Vantagens

- Geração rápida de código React/TypeScript de qualidade elevada a partir de descrições em linguagem natural.
- Integração automática com GitHub — cria repositório e efetua commits por cada alteração aceite.
- Interface de pré-visualização em tempo real — permite ver o resultado imediatamente sem necessidade de configurar ambiente local.
- Suporte nativo a shadcn/ui, Radix UI e Tailwind CSS v4, stack moderna e profissional.
- Capacidade de iterar sobre o design e funcionalidades com base em feedback escrito.

Dificuldades

- Necessidade de prompts precisos e detalhados para obter resultados alinhados com a visão pretendida — prompts vagos produzem código genérico.
- A plataforma possui limites de uso no plano gratuito, limitando o número de gerações disponíveis por dia.
- Alterações complexas (refactoring de arquitetura, lógica de estado partilhado) requerem prompts muito específicos e por vezes múltiplas iterações.
- O código gerado, embora funcional, pode conter padrões não otimizados que requerem revisão manual.

3.2. Exemplos de Prompts Utilizados

Os seguintes prompts (traduzidos e adaptados) foram utilizados ao longo do desenvolvimento no Lovable.dev:

Prompt	Resultado obtido
Cria-me um site que serve para gestão de tarefas com um design simples e estratégico dirigido a estudantes universitários	Construção TaskTide: sidebar de cadeiras, lista de tarefas agrupadas com prioridades/estado, painel de próximas datas e toggle PT/EN — tudo guardado localmente no teu navegador.
Faz o site mais apelativo, mantem o minimalismo mas dá um pouco mais de personalidade adicionando alguns ícones alusivos simples e de fácil interpretação	Adição de ícones discretos e intuitivos (graduação, marcador, bandeira de prioridade, calendário, idiomas, bússola, lista) mantendo o minimalismo estrutural.

3.3. Limitações das Plataformas e da Integração CI/CD

- O Lovable.dev não suporta geração de testes unitários automatizados — os testes têm de ser escritos manualmente ou por outra ferramenta de IA.
- A plataforma gera um projeto com template fixo (TanStack Start), o que limita a escolha da stack base.
- O GitHub Actions no plano gratuito tem minutos mensais limitados, o que pode ser restritivo em equipas com muitos commits.
- O Vercel, no plano gratuito, limita o número de deployments simultâneos e o bandwidth mensal.
- A ausência de testes end-to-end automatizados no pipeline significa que erros de UI só são detetados manualmente.

4. GitHub, Agile e CI/CD

4.1. GitHub — Versionamento e Colaboração

O repositório GitHub foi criado automaticamente pelo Lovable.dev no início do projeto, com o nome tasktide-uni. A gestão do código foi realizada com as seguintes práticas:

- Commits atômicos por funcionalidade, gerados pelo Lovable a cada alteração aceite na plataforma.
- Branches separadas por feature/membro — branch main protegida com revisão obrigatória via Pull Request.
- Pull Requests com descrição das alterações e revisão por pelo menos um elemento do grupo antes de merge.
- Issues no GitHub para registo de bugs encontrados durante o desenvolvimento e testes.

O repositório inclui o presente relatório em formato PDF, conforme requisitado no enunciado.

4.2. Gestão Agile / Scrum — GitHub Projects

A gestão do projeto foi realizada através do GitHub Projects, com um board Kanban organizado em colunas: Backlog, Sprint Backlog, Em Progresso, Em Revisão e Concluído. Foram realizadas 2 sprints:

Sprint	Período
Criar site	Semana 1–2
Deploy	Semana 2–3
Relatório	Semana 2-3

4.3. Pipeline de CI/CD

Foi implementado um pipeline de CI/CD com duas fases distintas:

CI — GitHub Actions

A cada push ou Pull Request para a branch main, o GitHub Actions executa automaticamente:

- Instalação de dependências (bun install).
- Verificação de lint (bun run lint) — detecção de erros de código e code smells.
- Build de produção (bun run build) — validação de que o projeto compila sem erros.

O pipeline falha e bloqueia o merge se qualquer um destes passos não for concluído com sucesso, garantindo a qualidade do código na branch principal.

CD — Vercel

O Vercel está ligado ao repositório GitHub com deploy automático configurado:

- Cada merge na branch main despoleta automaticamente um novo deployment no Vercel.
- Cada Pull Request gera um deployment de preview com URL único, permitindo testar alterações antes do merge.
- O Vercel detecta automaticamente o framework (Vite/React) e configura o processo de build sem configuração manual adicional.

5. Prints e Evidências

Nota: As capturas de ecrã das funcionalidades, do Scrum Board no GitHub Projects, do repositório GitHub, do pipeline CI/CD e do deploy online devem ser inseridas nesta secção. Recomenda-se organizar as evidências pelas subsecções abaixo.

5.1. Aplicação em Funcionamento



TAREFAS ATIVAS / ACTIVE TASKS

☒

Resolver Lista de Exercícios ...

CALC

ALTA

EM CURSO

PRAZO
18 Jun

☒

Implementar Árvore Bin...

PROG

MÉDIA

EM CURSO

PRAZO
Hoje, 23:59

☐

Entrega de Projeto: Mecânica

FISQ

ALTA

A FAZER

PRAZO
23 Jun

☐

Exame Intermédio Cálculo

CALC

ALTA

A FAZER

PRAZO
25 Jun

☐

Apresentação Seminário

HIST

MÉDIA

A FAZER

PRAZO
30 Jun

☒

Leitura: O Renascimento em ...

HIST

BAIXA

CONCLUÍDO

PRAZO
16 Jun

PT | EN

5.2. GitHub — Repositório e Colaboração

Criou dashboard Scholar OS

cc764a1

<>

lovable-dev[bot] committed 4 days ago

Changes

28245ea

<>

lovable-dev[bot] committed 4 days ago

Changes

bbb3054

<>

lovable-dev[bot] committed 4 days ago

Changes

10e2a6b

<>

lovable-dev[bot] committed 4 days ago

Changes

cf18a66

<>

lovable-dev[bot] committed 4 days ago

Changes

745276d

<>

lovable-dev[bot] committed 4 days ago

Changes

c16eac7

<>

lovable-dev[bot] committed 4 days ago

Adicionou ícones ao site ...
 lovable-dev[bot] committed 2 days ago

Changes
 lovable-dev[bot] committed 2 days ago

Changes
 lovable-dev[bot] committed 2 days ago

Work in progress
 lovable-dev[bot] committed 2 days ago

Update team members in README.md
 DavidMalafaya authored yesterday · ✓ 1 / 1

Create README.md (overview and details) ...
 ritamiranda001 authored 2 days ago

Add Vercel configuration file vercel.json
 ritamiranda001 authored 14 hours ago · ✓ 1 / 1

Add nitro preset for Vercel deployment ...
 ritamiranda001 authored 14 hours ago · ✓ 1 / 1

Link Repositório: <https://github.com/ritamiranda001/tasktide-uni.git>

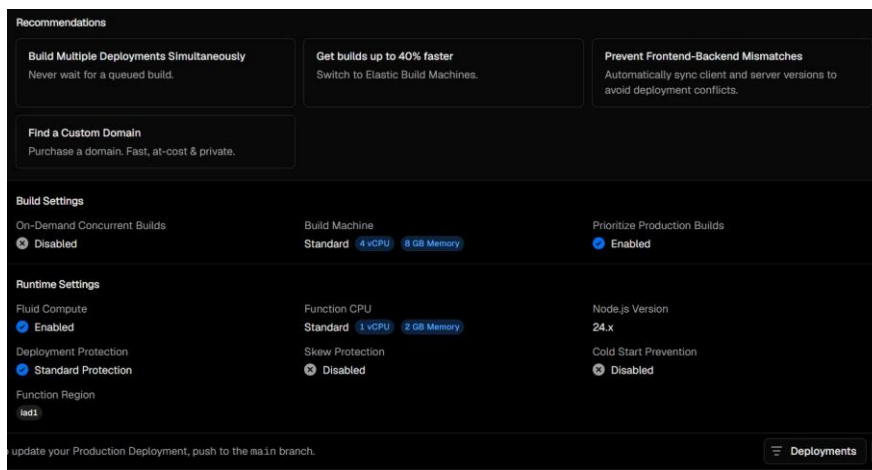
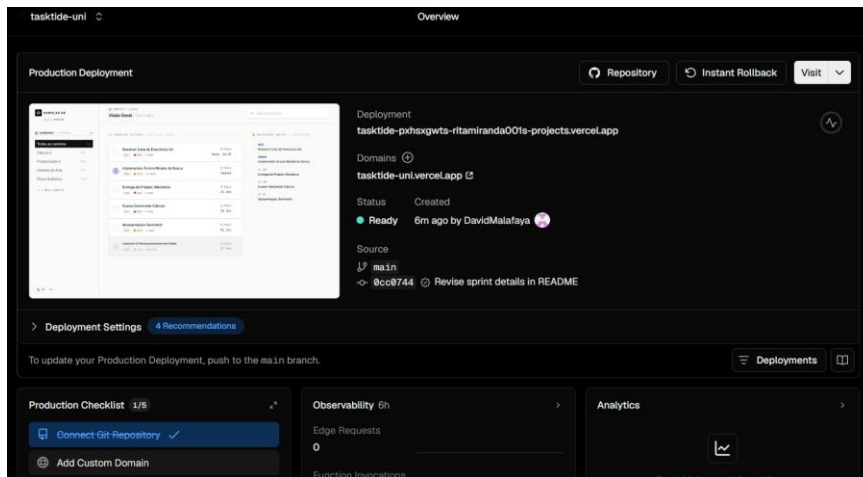
5.3. Scrum Board — GitHub Projects

☐  **Relatório**
#3 · by DavidMalafaya was closed now

☐  **Deploy**
#2 · by DavidMalafaya was closed 24 minutes ago

☐  **Criar Site**
#1 · by DavidMalafaya was closed 15 hours ago

5.4. Pipeline CI/CD — GitHub Actions e Vercel



5.5. Links do Projeto

Recurso	Link
Aplicação Online (Vercel)	https://tasktide-uni.vercel.app
Repositório GitHub	https://github.com/ritamiranda001/tasktide-uni.git

6. Conclusão e Análise Crítica

6.1. Processo e Metodologias de Desenvolvimento

A adoção de uma metodologia Agile/Scrum simplificada, com três sprints bem definidos e um backlog priorizado, revelou-se adequada para a dimensão e complexidade do projeto. A divisão de tarefas entre os membros da equipa (desenvolvimento do site, deploy/CI/CD e documentação) permitiu um trabalho em paralelo eficiente. A utilização do GitHub Projects - issues como ferramenta de gestão Scrum demonstrou a viabilidade de gerir projetos académicos com ferramentas profissionais de uso gratuito.

A metodologia de vibe coding com o Lovable.dev representou uma abordagem de desenvolvimento fundamentalmente diferente do processo tradicional: em vez de escrever código linha a linha, o desenvolvimento consistiu na formulação de requisitos e refinamentos em linguagem natural, com validação visual imediata do resultado. Esta abordagem é particularmente eficaz na fase de prototipagem, onde a velocidade de iteração é mais importante do que o controlo fino sobre cada detalhe de implementação.

6.2. Potencial das Ferramentas de IA

As ferramentas de IA para geração de código demonstraram um potencial significativo na aceleração do desenvolvimento de protótipos funcionais. O Lovable.dev conseguiu gerar, em poucas horas, código React/TypeScript de qualidade que teria levado vários dias a produzir manualmente, incluindo componentes complexos como o diálogo de edição com calendário, a sidebar de cadeiras com CRUD completo e a lógica de persistência em localStorage.

A IA revelou-se especialmente útil para tarefas repetitivas (geração de tipos TypeScript, variantes de componentes UI) e para sugerir padrões de implementação modernos (hooks personalizados, composição de componentes). Por outro lado, a IA mostrou limitações na compreensão de contexto complexo — requisitos que envolvem múltiplos componentes interdependentes por vezes resultaram em código inconsistente que necessitou de revisão manual.

6.3. Limitações Encontradas

- A plataforma Lovable.dev tem limites de uso no plano gratuito, o que condicionou o ritmo de desenvolvimento e obrigou a planear cuidadosamente os prompts para maximizar cada geração.
- A ausência de testes automatizados no projeto (unitários e end-to-end) é uma lacuna relevante para um produto em produção, embora aceitável para um protótipo académico.
- A persistência exclusiva em localStorage significa que os dados não são partilháveis entre dispositivos nem entre utilizadores, limitando os casos de uso colaborativo.
- O vibe coding tende a gerar código com algum "over-engineering" em componentes simples — foram identificados alguns padrões mais complexos do que o necessário para as funcionalidades implementadas.
- Os commits feitos via Lovable.dev aparecem no histórico como lovable-dev, o que dificultou na atribuição de contribuições individuais no GitHub

6.4. Vantagens do Trabalho Colaborativo

A distribuição de responsabilidades claras entre os membros da equipa — desenvolvimento da interface, configuração de deploy/CI/CD e documentação — permitiu que cada elemento se focasse na sua área de competência, resultando num produto mais completo do que seria possível individualmente. A utilização de Pull Requests como mecanismo de revisão de

código introduziu uma camada de controlo de qualidade que identificou e corrigiu vários problemas antes de chegarem à branch principal.

A comunicação assíncrona via Issues e comentários em Pull Requests no GitHub demonstrou ser um modelo de colaboração eficaz, especialmente considerando as diferentes disponibilidades horárias dos membros da equipa.

6.5. Impacto do CI/CD no Desenvolvimento

A implementação do pipeline CI/CD teve um impacto positivo e imediato na qualidade do processo de desenvolvimento. A automatização de lint e build a cada push eliminou a situação frequente de "funciona na minha máquina" — todos os membros da equipa tinham garantia de que o código na branch main estava sempre em estado deployável. O deploy automático para preview no Vercel a cada Pull Request permitiu validar visualmente alterações sem necessidade de configurar o projeto localmente, agilizando significativamente o ciclo de revisão e aprovação.

O CI/CD transformou o GitHub de um simples repositório de código numa plataforma de entrega contínua, onde cada commit é automaticamente testado, validado e publicado — uma prática que, uma vez adotada, é difícil de prescindir em projetos futuros.

7. Bibliografia

GitHub. (2024). GitHub Actions Documentation. <https://docs.github.com/en/actions>

GitHub. (2024). GitHub Projects — Planning and tracking with Projects. <https://docs.github.com/en/issues/planning-and-tracking-with-projects>

Lovable. (2024). Lovable.dev — Build software products with AI. <https://lovable.dev>

Meta. (2024). React Documentation (v19). <https://react.dev>

Vercel. (2024). Vercel Documentation — Deployments. <https://vercel.com/docs/deployments/overview>